

LAPORAN TUGAS
ALJABAR LINEAR (PRAKTIKUM)
MODUL 1: PENGENALAN RUST DAN DASAR KOMPUTASI VEKTOR

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Aljabar Linear (Praktikum)



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	4
BAB II SETUP ENVIRONMENT DAN VERIFIKASI TOOLCHAIN	5
2.1 Deskripsi Program	5
2.2 Konfigurasi Toolchain	5
2.3 Kode Program	6
2.4 Output Program	7
2.5 Lesson Learnt	7
BAB III DASAR SINTAKS DAN SISTEM TIPE RUST	8
3.1 Deskripsi Program	8
3.2 Definisi Data yang Digunakan	8
3.3 Kode Program	8
3.4 Penjelasan Proses	9
Bagian 1: Variable Binding dan Immutability	9
Bagian 2: Operasi Aritmetika i32 dan f64	10
Bagian 3: Array dan Indexing	10
Bagian 4: Perulangan dan Percabangan	10
3.5 Output Program	10
3.6 Lesson Learnt	12
BAB IV VEKTOR DENGAN ARRAY DAN VEC	13
4.1 Deskripsi Program	13
4.2 Definisi Data yang Digunakan	13
4.3 Keterbatasan Array Native	13
4.4 Kode Program	14
4.5 Penjelasan Proses	15
Fungsi tambah dan skalar	15

Fungsi dot	16
Fungsi statistik	16
4.6 Output Program	16
4.7 Lesson Learnt	17
BAB V INTRODUKSI NALGEBRA UNTUK VEKTOR	18
5.1 Deskripsi Program	18
5.2 Definisi Data yang Digunakan	18
5.3 Kode Program	18
5.4 Penjelasan Proses	19
Pembuatan Vektor	19
Aritmetika Element-wise lewat Operator	19
Dot Product, Norma, dan Sudut	20
Fungsi Statistik	20
Latihan Terapan	20
5.5 Output Program	21
5.6 Perbandingan dengan Implementasi Manual	23
5.7 Lesson Learnt	23
BAB VI PENGUJIAN OTOMATIS DAN KUALITAS KODE	24
6.1 Unit Test	24
6.2 Hasil Pengujian	24
6.3 Kualitas Kode	26
BAB VII KESIMPULAN	27

BAB I PENDAHULUAN

Rumus vektor gampang dihafal dan gampang salah dipakai kalau tidak pernah dijalankan mesin. Menuliskannya sebagai program memaksa tiap langkah menjadi eksplisit, dan hasil hitungannya bisa langsung diadu dengan hitungan tangan. Modul 1 memilih Rust, bukan Python dengan NumPy, karena Rust memberi performa setara C sekaligus jaminan memory safety yang diperiksa kompilator. Untuk kode numerik yang hasilnya dipercaya orang lain, kesalahan tipe dan kesalahan akses memori sebaiknya jatuh saat kompilasi, bukan saat program sudah setengah jalan memproses data. Sistem tipenya bahkan sanggup membawa dimensi vektor, sehingga lewat nalgebra penjumlahan dua vektor berbeda panjang ditolak sebelum program sempat lahir. Disiplin tipe, ownership, dan pengujian otomatis pun ikut terbentuk sejak modul pertama, sebelum kodenya sempat membesar.

Ruang lingkup pada bagian I.2 modul menetapkan empat sasaran. Toolchain rustup, cargo, dan rust-analyzer harus siap sampai sebuah proyek cargo terkompilasi tanpa warning, lalu dasar sintaks dan sistem tipe Rust dikuasai mulai dari `i32`, `f64`, `array`, `slice`, dan `Vec<T>` sampai struktur kontrol, fungsi, dan ownership. Sasaran ketiga merepresentasikan vektor dengan `array` dan `Vec` native lewat operasi manual berbasis iterator sekaligus menunjukkan batasnya terhadap operator matematika, sedangkan sasaran keempat membandingkannya dengan `crate nalgebra` yang menyediakan operator `overloading`, `dot product`, `norma`, dan fungsi statistik. Semuanya divalidasi memakai `cargo test`.

Vektor di $\mathbb{R}^n$ adalah sekumpulan komponen terurut $\mathbf{v} = (v_1, v_2, \dots, v_n)$, dan di memori ia sekadar blok nilai homogen yang berjajar. Penjumlahan bekerja per komponen, $\mathbf{u} + \mathbf{v} = (u_1+v_1, u_2+v_2, \dots, u_n+v_n)$, sedangkan perkalian skalar mengalikan setiap komponen sehingga $k*\mathbf{v} = (k*v_1, k*v_2, \dots, k*v_n)$. Dot product memampatkan dua vektor menjadi satu bilangan, $\mathbf{u} \cdot \mathbf{v} = u_1*v_1 + u_2*v_2 + \dots + u_n*v_n$. Bilangan tunggal itulah yang menjembatani aljabar dengan geometri lewat relasi $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$, sehingga sudut antara dua vektor terbaca sebagai $\theta = \arccos(\mathbf{u} \cdot \mathbf{v} / (|\mathbf{u}| |\mathbf{v}|))$. Norma atau panjang vektor didefinisikan sebagai $|\mathbf{v}| = \sqrt{\mathbf{v} \cdot \mathbf{v}}$, yakni dot product sebuah vektor dengan dirinya sendiri, dan norma itu pula yang menjadi penyebut pada relasi sudut tadi.

Laporan ini terbagi menjadi tujuh bab. BAB II memuat penyiapan lingkungan kerja dan Task 0, BAB III sampai BAB V membahas Task I hingga Task III secara berurutan, BAB VI merangkum 17 unit test yang dijalankan `cargo test`, dan BAB VII berisi kesimpulan beserta `lesson learnt`.

BAB II SETUP ENVIRONMENT DAN VERIFIKASI TOOLCHAIN

2.1 Deskripsi Program

Task 0 tidak menghitung apa pun. Yang diuji rantai perkakasnya, lewat sebuah proyek cargo bernama praktikum-al yang diberi dependensi nalgebra 0.35.0, lalu satu Vector3 dicetak untuk membuktikan crate itu benar terpasang.

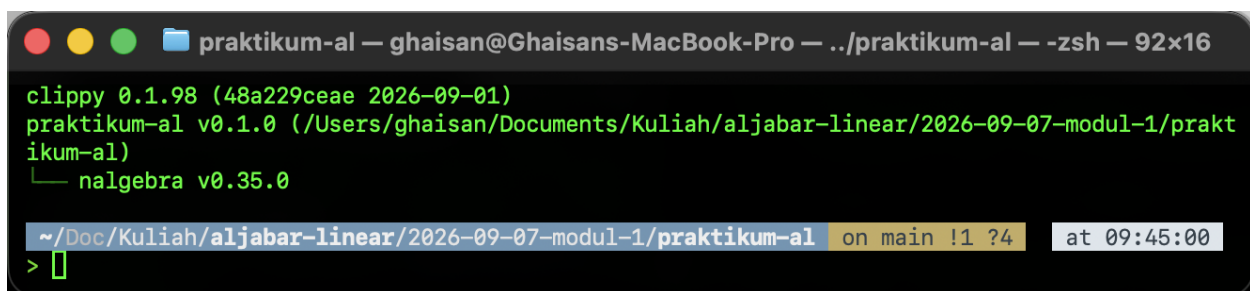
Pengecekan sekecil ini dikerjakan lebih dulu karena kegagalan build punya dua sumber yang gejalanya nyaris sama, yaitu kode yang salah dan toolchain yang rusak. Satu pesan error dari cargo bisa berarti salah sintaks, bisa juga berarti crates.io tidak terjangkau. Begitu Task 0 jalan, semua kegagalan sesudahnya pasti berasal dari kode sendiri. Alasan kedua menyangkut versi, karena API nalgebra berubah antar rilis minor dan edition 2024 memberlakukan aturan berbeda dari pendahulunya, jadi versi yang dipakai perlu dicatat sejak awal supaya angka pada bab berikutnya bisa ditelusuri ulang.

2.2 Konfigurasi Toolchain

Pemasangan dilakukan lewat rustup, skrip resmi dari sh.rustup.rs. Yang terpasang bukan rustc telanjang melainkan manajer toolchain, yang menarik channel stable berisi rustc, cargo, rustfmt, dan clippy sekaligus ke ~/.cargo/bin, sehingga direktori itu perlu masuk PATH. Pada MacBook Pro M2 Pro yang dipakai, target aarch64-apple-darwin dipilih otomatis dari deteksi arsitektur host. Cargo sendiri memegang dua peran, yakni build system yang memanggil rustc dengan flag yang benar dan menyimpan hasil antara di direktori target, sekaligus package manager yang membaca dependensi deklaratif di Cargo.toml lalu mengunduhnya dari crates.io berikut dependensi transitifnya. Menambahkan nalgebra cukup satu baris di bawah [dependencies], sisanya diurus cargo build.

```
[package]
name = "praktikum-al"
version = "0.1.0"
edition = "2024"

[dependencies]
nalgebra = "0.35.0"
```



Gambar 2.1: Verifikasi versi rustc, cargo, clippy, dan nalgebra

Versi yang terpasang adalah rustc 1.98.1, cargo 1.98.1, dan clippy 0.1.98, ketiganya dari channel stable yang sama. Versi nalgebra dibaca dari Cargo.lock, bukan dari Cargo.toml, karena manifest hanya menyatakan permintaan sedangkan berkas kunci mencatat apa yang benar dipakai, dan hasilnya 0.35.0. Modul mensyaratkan rustc minimal 1.75, jadi ambang itu terlampaui jauh.

2.3 Kode Program

Potongan berikut mengambil dua fungsi dari src/main.rs. Komentar identitas, tiga deklarasi mod, dan baris use nalgebra::Vector3 di atasnya dilewati.

```
fn task0() {
    println!("Hello, Aljabar Linear!");
    println!("Rust version check: OK");

    let v = Vector3::new(1.0, 2.0, 3.0);
    println!("nalgebra OK, contoh vektor: {v}");
}

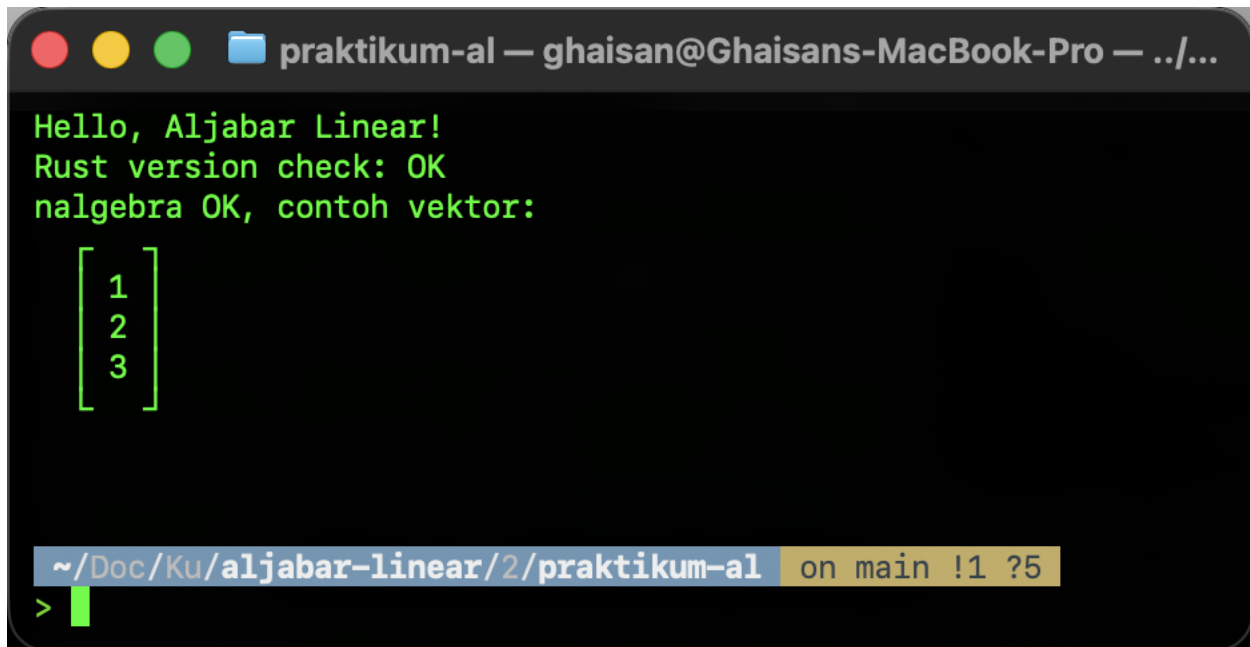
fn main() {
    match std::env::args().nth(1).as_deref() {
        Some("0") => task0(),
        Some("1") => task1::jalankan(),
        Some("2") => task2::jalankan(),
        Some("3") => task3::jalankan(),
        _ => {
            task0();
            task1::jalankan();
            task2::jalankan();
            task3::jalankan();
        }
    }
}
```

Isi task0() cuma dua panggilan println! lalu Vector3::new(1.0, 2.0, 3.0), yang tipe elemennya disimpulkan f64 dari bentuk literal desimalnya. Penulisan {v} di dalam string format adalah inline format argument, cara ringkas memanggil implementasi Display milik nalgebra tanpa menyebut variabelnya lagi sebagai argumen terpisah.

Fungsi main() tidak langsung memanggil keempat task, melainkan mencocokkan argumen baris perintah lebih dulu. Ekspresi std::env::args() mengembalikan iterator dengan nama program pada indeks nol, sehingga nth(1) mengambil argumen pertama yang diketik pengguna dan menghasilkan Option<String>. Panggilan as_deref() mengubahnya menjadi Option<&str> supaya bisa dicocokkan dengan pola Some("0") sampai Some("3"), sebab literal string di dalam pola bertipe &str dan bukan String. Tanpa as_deref() kompilator langsung menolak dengan galat mismatched types, dan di situlah sistem tipe Rust pertama kali terasa galak. Arm _ menangkap sisanya termasuk kondisi tanpa argumen, dipakai supaya tangkapan layar tiap bab tidak tercampur keluaran task lain.

2.4 Output Program

Hasil eksekusi cargo run -- 0 pada terminal ditampilkan berikut ini.



```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../...  
Hello, Aljabar Linear!  
Rust version check: OK  
nalgebra OK, contoh vektor:  

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$
  
~/.Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5  
>
```

Gambar 2.2: Hasil cargo run untuk Task 0

Dua baris pertama, Hello, Aljabar Linear! dan Rust version check: OK, membuktikan rustc berhasil mengompilasi berkas dan binary-nya benar dieksekusi. Baris ketiga menampilkan vektor yang dicetak nalgebra sebagai matriks kolom berbingkai, dengan nilai 1, 2, dan 3 tersusun dari atas ke bawah. Angka tampil tanpa bagian desimal walaupun tipenya f64, karena implementasi Display milik nalgebra memangkas nol di belakang koma. Bentuk kolom itu menyiratkan Vector3 sebenarnya matriks tiga baris satu kolom.

2.5 Lesson Learnt

Memastikan toolchain benar sebelum menulis kode ternyata bukan formalitas. Saat menyiapkan environment, rustup sempat dijalankan di dua jendela terminal sekaligus, satu untuk update dan satu untuk memasang komponen tambahan. Keduanya menulis ke direktori ~/.rustup yang sama, metadata toolchain jadi rusak, dan setiap pemanggilan cargo sesudahnya berhenti dengan galat missing manifest. Yang bikin bingung, pesan itu sama sekali tidak menyinggung penyebabnya. Perbaikannya menghapus toolchain stable lalu memasangnya ulang dari nol. Pelajarannya sederhana, rustup tidak aman dijalankan paralel.

Hal kedua, baris nalgebra = "0.35.0" pada manifest menyatakan rentang versi yang diterima, bukan satu titik pasti. Cargo.lock menutup celah itu dengan mencatat versi persis sampai ke dependensi transitif, jadi angka pada bab berikutnya bisa direproduksi.

BAB III DASAR SINTAKS DAN SISTEM TIPE RUST

3.1 Deskripsi Program

Kode Task I ditaruh di berkas terpisah `src/task1.rs`, dideklarasikan lewat `mod task1`; pada `main.rs`, lalu dipanggil dengan `task1::jalankan()`. Isinya empat bagian eksplorasi yang dicetak berurutan dan dipisahkan garis pembatas oleh fungsi bantu `judul()`. Dua fungsi pendukung ikut ditulis, yaitu `hampir_sama()` untuk membandingkan dua nilai `f64` dan `klasifikasi()` untuk menentukan sebuah nilai genap atau ganjil, keduanya dipisah dari `jalankan()` supaya bisa dipanggil ulang dari modul `tests`.

Sasarannya fondasi bahasa sebelum masuk ke komputasi vektor, mulai dari cara Rust mengikat nilai ke nama sampai perulangan yang menggabungkan iterasi dengan percabangan. Konstanta `EPSILON` ditambahkan di luar daftar minimal modul karena membandingkan bilangan pecahan dengan operator sama dengan gampang meleset.

3.2 Definisi Data yang Digunakan

Ada empat binding bertipe dasar, yaitu `n` bertipe `i32` bernilai 10, `phi` bertipe `f64` bernilai 3.14159, `aktif` bertipe `bool` bernilai `true`, dan `nama` bertipe `&str` berisi "Vektor". Tabel 3 modul mencatat `i32` mendukung `+` `-` `*` `/` `%` beserta operasi bitwise, `f64` menambahkan method `powf`, `sqrt`, dan `abs`, `bool` hanya mengenal `&&`, `||`, serta `!`, sedangkan `&str` terbatas pada slicing dan penggabungan lewat `format!`. Ketiga method `f64` tadi terpakai semua di berkas ini.

Data numeriknya disimpan pada `v` bertipe `[f64; 5]` berisi 10.0 sampai 50.0, dan panjang array ikut menjadi bagian dari tipenya sehingga ukurannya pasti saat kompilasi. Dari array itu diambil potongan `&v[1..4]` bertipe `&[f64]`. Konstanta `EPSILON` dideklarasikan di lingkup modul sebagai `const EPSILON: f64 = 1e-10;`, dan berbeda dari `let`, `const` wajib beranotasi tipe serta nilainya harus bisa dihitung saat kompilasi. Angka `1e-10` jauh lebih besar dari galat pembulatan pada operasi sederhana, tetapi masih cukup kecil untuk menolak dua bilangan yang memang berbeda.

3.3 Kode Program

Potongan berikut memuat konstanta, dua fungsi pendukung, dan bagian isi `jalankan()` yang dibahas pada subbab setelahnya. Baris `// ...` menandai perintah cetak serta dua bagian sisanya.

```
/// Ambang toleransi buat membandingkan dua f64.
const EPSILON: f64 = 1e-10;

/// Perbandingan float pakai == gampang meleset karena galat pembulatan,
/// jadi selisihnya yang dicek terhadap EPSILON.
fn hampir_sama(a: f64, b: f64) -> bool {
    (a - b).abs() < EPSILON
}
```



```

fn klasifikasi(nilai: f64) -> &'static str {
    if nilai as i64 % 2 == 0 {
        "genap"
    } else {
        "ganjil"
    }
}

pub fn jalankan() {
    let n: i32 = 10;
    // 3.14159 mendekati PI, tanpa atribut ini clippy menolak
    #[allow(clippy::approx_constant)]
    let phi: f64 = 3.14159;
    let aktif: bool = true;
    let nama: &str = "Vektor";
    // ...

    // binding default immutable, mau diubah nilainya harus ditandai mut dulu
    let tetap = 5;
    let mut berubah = 5;
    berubah += 3;
    // ...

    let (a, b) = (7.0f64, 2.0f64);
    println!("a / b = {}", a / b);
    // pembagian antar i32 membuang bagian pecahannya, bukan dibulatkan
    println!("7 / 2 (i32) = {}", 7 / 2);
    // sqrt cuma ada di tipe float, integer harus di-cast dulu
    println!("9.0.sqrt() = {}", 9.0f64.sqrt());
    // ...
}

```

3.4 Penjelasan Proses

Bagian 1: Variable Binding dan Immutability

Anotasi tipe pada empat binding pertama sebenarnya bisa dihapus karena Rust sanggup menyimpulkannya, tetapi ditulis eksplisit supaya tipe yang dibahas terbaca langsung. Yang tidak bisa dihapus justru `#[allow(clippy::approx_constant)]` di atas `phi`. Clippy punya lint `approx_constant` yang mengendus literal pecahan bernilai dekat dengan konstanta matematika bawaan lalu menyarankan `std::f64::consts::PI`, dan 3.14159 persis masuk rentang deteksinya, padahal modul meminta angka itu ditulis apa adanya. Atributnya dipasang tepat di atas satu pernyataan yang bermasalah, bukan di kepala berkas, supaya lint yang sama tetap hidup untuk baris lain. Tanpa atribut itu cargo clippy `--all-targets` mengeluarkan peringatan dan syarat clippy bersih langsung gugur. Pasangan tetap dan berubah menutup bagian ini, keduanya berangkat dari 5 tetapi hanya yang bertanda `mut` yang boleh menerima `+= 3` dan berakhir tercetak 8.

Bagian 2: Operasi Aritmetika i32 dan f64

Pada f64 hasilnya lurus saja, a / b memberi 3.5, $a \% b$ memberi 1, dan $a.\text{powf}(b)$ memberi 49, dua angka terakhir tercetak tanpa koma desimal karena Display untuk f64 membuang pecahan bernilai nol. Yang menarik ada di $7 / 2$ yang menghasilkan 3, bukan 3.5. Literal 7 dan 2 tidak beranotasi maupun bersufiks, jadi compiler memilih i32 sebagai tipe integer bawaan, dan pembagian antar i32 menghasilkan i32 lagi sehingga bagian pecahannya dipotong ke arah nol, bukan dibulatkan. Setengah yang hilang di titik itu justru muncul lagi sebagai $7 \% 2$ yang bernilai 1. Baris `9.0f64.sqrt()` memakai sufiks tipe karena sqrt cuma didefinisikan untuk f32 dan f64, sebab akar bilangan bulat umumnya bukan bilangan bulat sehingga tidak ada nilai balik yang masuk akal untuk i32, dan integer harus di-cast dulu dengan as f64.

Bagian 3: Array dan Indexing

Indexing berjalan seperti dugaan, `v[0]` bernilai 10 dan elemen terakhir diambil lewat `v[v.len() - 1]` karena Rust tidak mengenal indeks negatif. Potongan `&v[1..4]` adalah slice bertipe `&[f64]`, pinjaman read-only yang hanya menyimpan pointer elemen awal beserta panjangnya tanpa menyalin data. Rentangnya setengah terbuka, jadi yang terambil indeks 1, 2, dan 3.

Bagian 4: Perulangan dan Percabangan

Perulangan `for` nilai `in v` mengambil elemen by value karena f64 bertipe Copy dan array mengimplementasikan IntoIterator. Tiap nilai diteruskan ke `klasifikasi()` yang meng-cast nilai as i64 lebih dulu, sebab operator `%` pada f64 tetap mengembalikan f64 dan pemeriksaan genap ganjil jadi kurang bersih dibanding pada bilangan bulat. Sisa bagi terhadap 2 dievaluasi di dalam `if`, lalu cabang yang menang mengembalikan salah satu dari dua literal bertipe `&'static str`.

3.5 Output Program

Hasil eksekusi perintah `cargo run -- 1` tampak pada gambar berikut.

```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../pr...

Finished `dev` profile [unoptimized + debuginfo] target(s) in
0.01s
Running `target/debug/praktikum-al 1`

-----
Bagian 1: Variable Binding & Immutability
-----

n      (i32) = 10
phi    (f64) = 3.14159
aktif  (bool) = true
nama   (&str) = Vektor
tetap  (tanpa mut, nilainya kunci) = 5
berubah (pakai mut, setelah += 3) = 8
EPSILON (const) = 1e-10
0.1 + 0.2 hampir sama 0.3 ? = true

-----
Bagian 2: Operasi Aritmetika i32 dan f64
-----

a / b      = 3.5
a % b      = 1
a.powf(b)  = 49
7 / 2 (i32) = 3
```

Gambar 3.1a: Hasil cargo run untuk Task I, bagian atas

```
7 % 2 (i32) = 1
9.0.sqrt() = 3

-----
Bagian 3: Array & Indexing
-----

v      = [10.0, 20.0, 30.0, 40.0, 50.0]
v[0]   = 10
v[len - 1] = 50
&v[1..4] = [20.0, 30.0, 40.0]

-----
Bagian 4: Perulangan & Percabangan
-----

10 -> genap
20 -> genap
30 -> genap
40 -> genap
50 -> genap
3 -> ganjil
4 -> genap
7 -> ganjil

~/Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5
> █
```

Gambar 3.1b: Hasil cargo run untuk Task I, bagian bawah

Bagian pertama mencetak keempat binding apa adanya, lalu tetap yang bertahan di 5 melawan berubah yang menjadi 8, EPSILON tercetak $1e-10$ lewat format `{:e}`, dan `hampir_sama(0.1 + 0.2, 0.3)` bernilai `true`. Kontras yang dicari muncul di bagian kedua, $a / b = 3.5$ pada `f64` berhadapan dengan $7 / 2 = 3$ pada `i32`, ditemani $a \% b = 1$, $a.\text{powf}(b) = 49$, dan $9.0.\text{sqrt}() = 3$. Bagian ketiga menampilkan `v[0] = 10`, `v[len - 1] = 50`, dan `&v[1..4] = [20.0, 30.0, 40.0]`. Bagian keempat sempat pincang. Isi array `v` yang diminta modul kelipatan sepuluh semua, jadi kelimanya terbaca genap dan cabang `else` tidak pernah tersentuh, karena itu ditambahkan perulangan atas array `[3.0, 4.0, 7.0]` yang membuat 3 dan 7 tercetak ganjil sementara 4 tercetak genap, sehingga cabang ganjil ikut terbukti dieksekusi.

3.6 Lesson Learnt

Beda `i32` dan `f64` bukan soal ada tidaknya koma desimal, melainkan soal operasi apa yang tersedia dan bagaimana hasilnya diperlakukan. Tipe `i32` menyimpan bilangan bulat secara eksak tetapi pembagiannya memotong, sementara `f64` menyediakan `sqrt`, `powf`, dan `abs` dengan bayaran nilai yang cuma hampiran. Sifat hampiran itu yang membuat $0.1 + 0.2$ tidak persis sama dengan 0.3 , jadi setiap perbandingan `f64` di sini lewat `hampir_sama()` dengan ambang EPSILON, bukan operator sama dengan. Rust juga tidak mempromosikan tipe secara otomatis seperti C, mencampur `i32` dan `f64` tanpa `cast` eksplisit langsung gagal kompilasi.

Immutability sebagai default membalik kebiasaan dari bahasa lain, sebab yang perlu ditulis justru izin untuk berubah, bukan larangan berubah. Aturan itu tidak menambah biaya runtime sedikit pun, tetapi hasilnya compiler bisa memastikan sebuah nilai tidak berubah diam-diam di tengah perhitungan.

Penolakan $a + b$ atas dua array bukan kelalaian `standard library`, melainkan keputusan desain, sebab trait `Add` memang tidak diimplementasikan untuk `[T; N]`. Array cuma wadah elemen sejenis berpanjang tetap, tanpa makna matematis yang melekat, jadi tidak ada dasar untuk menebak apakah $+$ berarti penjumlahan *element-wise* atau penggabungan. Akibatnya operasi itu ditulis manual lewat iterator, dan itulah yang dikerjakan pada Task II. Keterbatasan yang sama menjadi alasan *nalgebra* dipakai pada Task III, karena tipe vektornya sudah membawa operator aritmetika lengkap dengan pemeriksaan dimensi saat kompilasi.

BAB IV VEKTOR DENGAN ARRAY DAN VEC

4.1 Deskripsi Program

Dua vektor tiga dimensi disimpan sebagai array `f64` bawaan, lalu dijumlahkan, dikalikan skalar, dan dihitung hasil kali titiknya tanpa satu `crate` pun. Isi `u` adalah `[1.0, 2.0, 3.0]` dan isi `v` adalah `[4.0, 5.0, 6.0]`, persis angka yang dipakai modul pada poin 1.

Semua fungsi hitung berdiri di atas iterator, bukan loop indeks. Pola yang berulang adalah `iter()` untuk membuka aliran elemen, `zip()` untuk memasangkan dua vektor, `map()` untuk menghitung per pasangan, lalu `collect()` atau `sum()` sebagai penutup rantai. Di luar aritmetika, program juga meringkas satu vektor menjadi jumlah, rata-rata, nilai maksimum, dan nilai minimum. Enam unit test di modul `#[cfg(test)]` berkas yang sama mengunci hasilnya, termasuk satu test yang sengaja memancing `panic`.

4.2 Definisi Data yang Digunakan

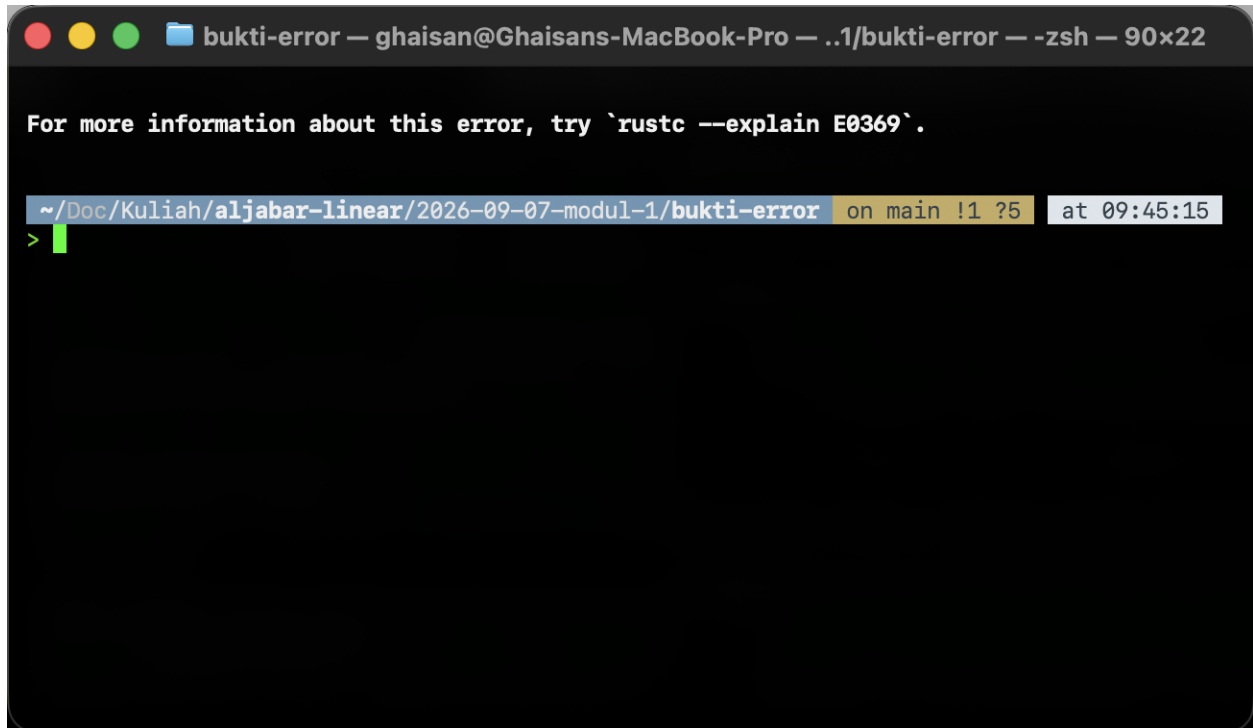
Tipe `[f64; 3]` adalah array panjang tetap, dan panjangnya ikut menjadi bagian dari tipenya. Ukurannya diketahui saat kompilasi sehingga isinya duduk di `stack` tanpa alokasi `heap`. Akibatnya `[f64; 3]` dan `[f64; 4]` adalah dua tipe berbeda yang tidak bisa saling dipertukarkan.

Supaya fungsinya tidak terkunci pada satu panjang, seluruh parameter di `src/task2.rs` bertipe `&[f64]`, yaitu `slice`, sebuah `borrow read-only` yang isinya cuma pointer dan panjang. Nilai kembaliannya `Vec<f64>` karena `collect()` harus menyerahkan data yang dimiliki pemanggil, sedangkan `slice` tidak pernah memiliki apa pun. Array masuk ke fungsi lewat coercion `&u` yang berlangsung otomatis.

4.3 Keterbatasan Array Native

Poin 2 modul meminta bukti bahwa `let w = u + v;` ditolak kompilator, lengkap dengan pesan galat aslinya. Operator `+` di Rust bukan hak istimewa yang otomatis melekat pada sembarang tipe, melainkan gula sintaksis untuk `trait std::ops::Add`, dan array bawaan tidak pernah mengimplementasikan trait itu. Baris tersebut diaktifkan sebentar untuk merekam keluaran `rustc 1.98.1`, lalu dikembalikan menjadi komentar supaya proyeknya tetap bisa dibangun.

Pesannya pendek dan tidak ambigu, yaitu `error[E0369]: cannot add [f64; 3] to [f64; 3]`. Kompilator bahkan menggarisbawahi kedua operand dan menuliskan tipenya di bawah tanda `+`, jadi tidak ada tebak-tebakan soal apa yang salah.



Gambar 4.1: Galat E0369 saat mencoba menjumlahkan dua array secara langsung

Python tidak menolak apa pun pada kasus serupa. Ekspresi `[1, 2, 3] + [4, 5, 6]` dijalankan lewat `list.__add__` dan menghasilkan `[1, 2, 3, 4, 5, 6]`, yaitu konkatenasi enam elemen, bukan penjumlahan tiga elemen. Tidak ada peringatan, tidak ada exception, program jalan terus, dan kesalahannya baru diketahui kalau ada yang curiga kenapa panjang vektornya berlipat. Rust memilih menggagalkan kompilasi ketimbang menebak maksud penulis, dan galat sedini ini jauh lebih murah daripada angka keliru yang lolos sampai ke hasil akhir. Penolakan inilah yang menjadi alasan Task III berpindah ke nalgebra, karena di sana `+` memang sudah diberi makna aljabar linear yang benar.

4.4 Kode Program

Potongan di bawah memuat inti perhitungan `src/task2.rs` berupa fungsi aritmetika, dot, statistik, dan satu test yang paling mewakili. Bagian yang tidak dibahas ditandai `// ...`

```
pub fn tambah(u: &[f64], v: &[f64]) -> Vec<f64> {
    assert_eq!(u.len(), v.len(), "panjang kedua vektor harus sama");
    u.iter().zip(v).map(|(a, b)| a + b).collect()
}

pub fn skalar(k: f64, u: &[f64]) -> Vec<f64> {
    u.iter().map(|a| k * a).collect()
}

pub fn dot(u: &[f64], v: &[f64]) -> f64 {
    assert_eq!(u.len(), v.len(), "panjang kedua vektor harus sama");
    u.iter().zip(v).map(|(a, b)| a * b).sum()
}
```

```

}

pub fn jumlah(u: &[f64]) -> f64 {
    u.iter().sum()
}

pub fn rata(u: &[f64]) -> f64 {
    jumlah(u) / u.len() as f64
}

pub fn maks(u: &[f64]) -> f64 {
    u.iter().copied().fold(f64::NEG_INFINITY, f64::max)
}

pub fn mini(u: &[f64]) -> f64 {
    u.iter().copied().fold(f64::INFINITY, f64::min)
}

// ...

#[cfg(test)]
mod tests {
    use super::*;

    // ...

    #[test]
    #[should_panic(expected = "panjang kedua vektor harus sama")]
    fn panjang_beda_ditolak() {
        dot(&[1.0, 2.0, 3.0], &[1.0, 1.0]);
    }
}

```

4.5 Penjelasan Proses

Fungsi tambah dan skalar

Badan fungsi tambah cuma satu baris kerja. Metode `iter()` mengeluarkan `&f64` karena slice-nya sekadar dipinjam, `zip` menempelkan `v` di sisinya, `map` menjumlahkan tiap pasangan, dan `collect()` membangun `Vec<f64>` baru. Anotasi tipe kembalianlah yang menentukan `collect()` harus mengumpulkan ke `Vec`, sebab metode itu generik atas tipe tujuannya dan tanpa petunjuk apa pun kompilator akan berhenti minta anotasi. Baris `assert_eq!(u.len(), v.len(), ...)` mengikuti Tabel 4 modul yang menyebut pemeriksaan dimensi array native dilakukan manual saat runtime. Tanpa penjaga itu `zip` berhenti di slice terpendek dan hasilnya salah tanpa suara. Fungsi skalar lebih ringan lagi, satu `map` tanpa `zip` karena operand vektornya cuma satu.

Fungsi dot

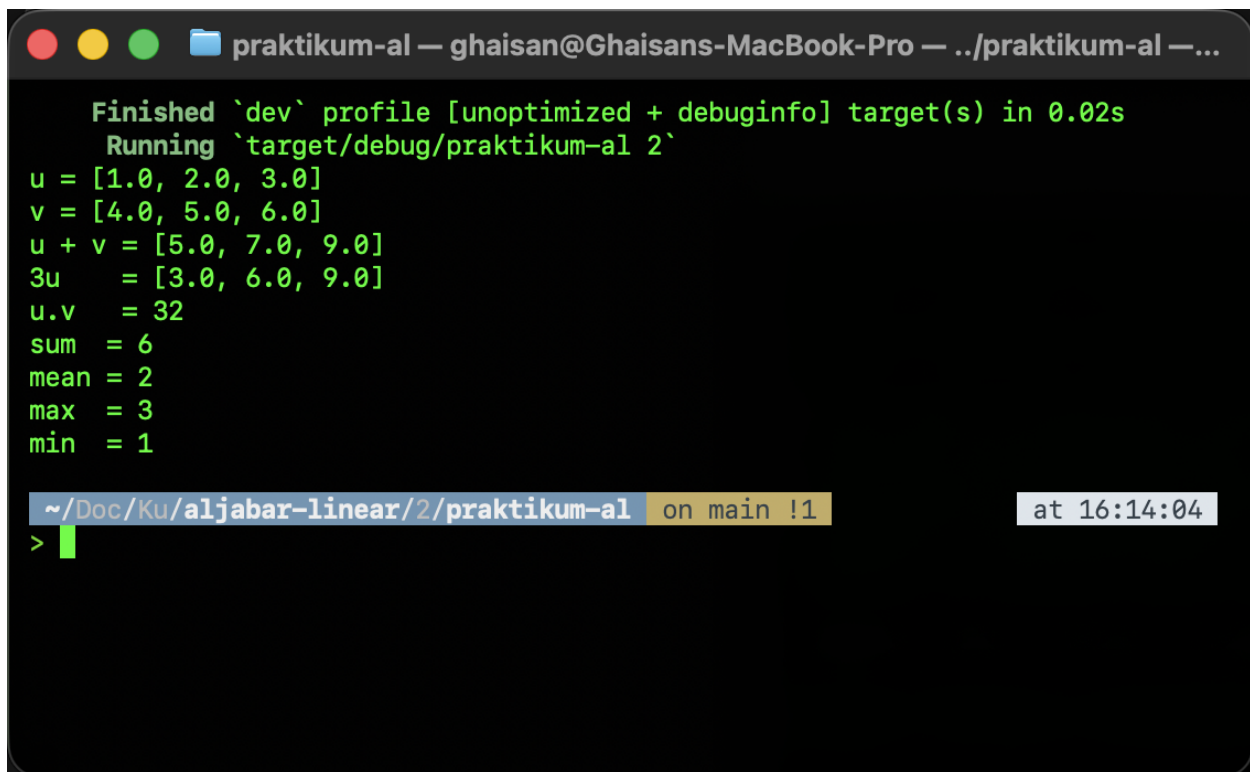
Alurnya identik dengan tambah sampai tahap map, bedanya penutupnya `sum()` dan bukan `collect()`. Aliran hasil perkalian diringkas menjadi satu `f64` tunggal sehingga tipe kembaliannya bukan koleksi. `assert_eq!` yang sama dipasang di sini, dan fungsi inilah yang dipakai test `panjang_beda_ditolak` untuk memastikan panic-nya memang terjadi beserta pesannya.

Fungsi statistik

Fungsi jumlah cukup memanggil `u.iter().sum()` karena `f64` sudah punya implementasi `Sum`, sedangkan rata membaginya dengan `u.len()` as `f64`, konversi eksplisit yang diwajibkan Rust sebab `len()` bertipe `usize`. Yang lebih menarik `maks` dan `mini`, keduanya tidak bisa memakai `Iterator::max` maupun `Iterator::min` karena kedua metode itu menuntut elemennya mengimplementasikan `Ord`. Tipe `f64` tidak mengimplementasikan `Ord` gara-gara `NaN` yang tidak pernah sama dengan dirinya sendiri, jadi urutannya tidak total dan yang tersedia hanya `PartialOrd`. Solusinya fold dengan `f64::max` dan `f64::min`, dua fungsi yang sudah mendefinisikan sendiri perlakuannya terhadap `NaN`. Nilai awal fold-nya `f64::NEG_INFINITY` untuk `maks` dan `f64::INFINITY` untuk `mini`, supaya elemen pertama apa pun langsung menggantikannya.

4.6 Output Program

Perintah `cargo run -- 2` menjalankan Task II saja.



```
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
Running `target/debug/praktikum-al 2`
u = [1.0, 2.0, 3.0]
v = [4.0, 5.0, 6.0]
u + v = [5.0, 7.0, 9.0]
3u      = [3.0, 6.0, 9.0]
u.v      = 32
sum      = 6
mean     = 2
max      = 3
min      = 1

~/.Doc/Ku/aljabar-linear/2/praktikum-al on main !1 at 16:14:04
>
```

Gambar 4.2: Hasil cargo run untuk Task II

Penjumlahan $u + v$ tercetak $[5.0, 7.0, 9.0]$ dan perkalian skalar $3u$ tercetak $[3.0, 6.0, 9.0]$, keduanya cocok dengan nilai acuan pada poin 3 modul. Hasil kali titik $u.v$ bernilai 32, sama dengan hitungan tangan $1*4 + 2*5 + 3*6$. Blok statistik memberi sum 6, mean 2, max 3, dan min 1 untuk vektor u . Seluruh proyek lulus cargo test dengan 17 passed 0 failed, sementara cargo build dan clippy tidak mengeluarkan satu warning pun.

4.7 Lesson Learnt

Menulis operasi vektor sendiri jelas lebih bertele-tele daripada mengetik $u + v$. Tiga operasi dasar menuntut tiga fungsi terpisah ditambah dua `assert_eq!` yang cuma mengulang hal yang idealnya dijamin tipe. Imbalannya galat yang tegas, bukan angka keliru yang lolos diam-diam.

Batasnya tetap ada. Kompilator menangkap penyalahgunaan operator, tetapi tidak menangkap ketidakcocokan dimensi, sebab `&[f64]` membuang informasi panjang dari tipenya sehingga panjang beda baru meledak saat runtime.

Untuk tiga sampai empat operasi pada vektor tiga elemen, pendekatan manual masih sepadan. Pustaka numerik baru perlu begitu operasinya bertambah jadi norma, proyeksi, dan perkalian matriks, dan itulah yang dijawab nalgebra pada Task III.

BAB V INTRODUKSI NALGEBRA UNTUK VEKTOR

5.1 Deskripsi Program

Operasi vektor yang sama seperti Task II dikerjakan ulang di sini, bedanya seluruh perhitungan diserahkan ke crate nalgebra 0.35.0. Program dipanggil lewat cargo run -- 3 dan masuk ke fungsi jalankan() pada berkas src/task3.rs.

Isinya mengikuti urutan modul, dari pembuatan vektor, aritmetika element-wise lewat operator, dot product bersama norma dan sudut, fungsi statistik, sampai dua latihan terapan. Berkas ini juga memuat enam unit test, satu di antaranya mengerjakan poin E.3 modul yaitu identitas $u \cdot u$ sama dengan kuadrat norma u . Perbandingan floating point memakai selisih mutlak terhadap konstanta EPS bernilai $1e-10$, kecuali pengujian element-wise yang membandingkan Vector3 secara utuh karena nilainya bulat dan terwakili persis dalam biner.

5.2 Definisi Data yang Digunakan

Vektor a dan b dibuat dengan `Vector3::new(1.0, 2.0, 3.0)` dan `Vector3::new(4.0, 5.0, 6.0)`. `Vector3<f64>` bukan tipe tersendiri melainkan alias untuk `SVector<f64, 3>`, jadi angka 3 ikut menjadi bagian dari tipe dan compiler yang memeriksanya. Pemanggilan `a.nrows()` tetap ada, tetapi nilainya konstanta sejak kompilasi, bukan hasil pengukuran buffer seperti `len()` pada slice.

Bentuk generik `SVector<f64, N>` muncul dua kali dengan N berbeda, yaitu `b4` sebagai `SVector<f64, 4>` yang diisi dari array literal, dan `w` sebagai `SVector<f64, 10>` yang diisi lewat `from_iterator`. `Iterator (1..=10)` menghasilkan `i32`, jadi `f64::from` dipakai sebelum masuk ke konstruktor. Latihan pertama memakai `Vector2<f64>` dengan $p = (3, 4)$ dan $q = (4, 0)$, angka pilihan modul karena p membentuk segitiga siku-siku 3-4-5. Seluruh data di task ini bertipe `f64`, sebab norma dan `acos` hanya masuk akal pada bilangan pecahan.

5.3 Kode Program

Potongan berikut memuat bagian yang dibahas pada subbab berikutnya, sisanya ditandai komentar titik tiga.

```
use nalgebra::{SVector, Vector2, Vector3};

pub fn jalankan() {
    // A. Pembuatan vektor
    let a = Vector3::new(1.0, 2.0, 3.0);
    let b4: SVector<f64, 4> = SVector::from([1.0, 2.0, 3.0, 4.0]);
    println!("a = {a}");
    println!("dimensi a: {}", a.nrows());
    println!("b4 = {b4}");
    println!("dimensi b4: {}", b4.nrows());

    // B. Aritmetika element-wise lewat operator
    let b = Vector3::new(4.0, 5.0, 6.0);
```

```
println!("a + b = {}", a + b);
println!("a - b = {}", a - b);
println!("2a = {}", 2.0 * a);
println!("a o b = {}", a.component_mul(&b)); // perkalian Hadamard

// C. Dot product, norma, dan sudut
let dot_ab = a.dot(&b);
let norm_a = a.norm();
// anotasi : f64 wajib di sini, tanpa itu tipe hasil bagi masih generik
// dan pemanggilan .acos() ditolak dengan E0282 type annotations needed
let cos: f64 = a.dot(&b) / (a.norm() * b.norm());
let sudut = cos.acos().to_degrees();
println!("a . b = {dot_ab}");
println!("||a|| = {norm_a}");
println!("sudut a dan b = {sudut} derajat");

// D. Fungsi statistik dan E. Latihan terapan p, q, w
// ...
}

#[cfg(test)]
mod tests {
    use super::*;
    const EPS: f64 = 1e-10;
    // E.3 identitas u . u = ||u||^2, pakai toleransi karena floating point
    #[test]
    fn dot_diri_sendiri_sama_dengan_kuadrat_norma() {
        let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0);
        assert!((a.dot(&a) - a.norm().powi(2)).abs() < EPS);
    }
    // ...
}
```

5.4 Penjelasan Proses

Pembuatan Vektor

Dua konstruktor dari Tabel 5 modul dipakai berdampingan. `Vector3::new` menerima tiga argumen langsung, sementara `SVector::from` menerima array literal sehingga dimensinya ikut dari panjang array. Cetakan `a.nrows()` dan `b4.nrows()` memberi 3 dan 4.

Aritmetika Element-wise lewat Operator

Di sinilah beda paling terasa dibanding Task II. nalgebra mengimplementasikan trait `Add`, `Sub`, dan `Mul` untuk tipe vektornya, jadi `a + b` dan `a - b` ditulis apa adanya tanpa fungsi pembantu, dan hasilnya `[5, 7, 9]` serta `[-3, -3, -3]`. Perkalian skalar ditulis `2.0 * a`, bukan `a * 2.0`, karena yang tersedia adalah implementasi `Mul` pada `f64` di sisi kiri, dan keluarannya `[2, 4, 6]`.

Baris terakhirnya memanggil `a.component_mul(&b)` yang menghasilkan `[4, 10, 18]`. Itu perkalian Hadamard, elemen ke elemen, hasilnya tetap vektor dan bukan skalar seperti dot product. Keduanya sama-sama mengalikan pasangan elemen, bedanya dot product menjumlahkan hasilnya sehingga 4 ditambah 10 ditambah 18 menjadi 32. Nama `component_mul` dipakai justru

untuk mencegah kekeliruan itu, sebab operator * antar dua vektor artinya ambigu.

Dot Product, Norma, dan Sudut

Pemanggilan `a.dot(&b)` menerima referensi, jadi `b` tidak dipindahkan kepemilikannya dan masih bisa dipakai di baris berikutnya. Hasilnya 32, norma `a` adalah akar 14 atau sekitar 3.742, dan sudut antara keduanya sekitar 12.93 derajat, wajar karena arah `a` dan `b` memang berdekatan di ruang tiga dimensi.

Baris `let cos: f64` tidak bisa ditulis tanpa anotasinya, dan ini jebakan yang benar-benar menghentikan kompilasi. Metode `dot()` dan `norm()` milik `nalgebra` dideklarasikan generik atas tipe skalar yang mengimplementasikan `trait numerik crate` tersebut, sehingga hasil pembagian `a.dot(&b) / (a.norm() * b.norm())` masih berupa parameter tipe yang belum ditentukan. Pada titik itu compiler belum tahu tipe konkretnya, dan karena `.acos()` bukan metode dari `trait generik` tadi melainkan metode inheren milik `f64`, pemanggilannya ditolak dengan galat `E0282 type annotations needed`. Menulis `: f64` pada `cos` menutup celah itu, dan begitu tipenya pasti, `acos()` ikut terselesaikan. Logikanya tidak berubah sedikit pun, yang kurang cuma satu keterangan untuk compiler.

Jebakan yang sama muncul lagi di modul unit test dengan gejala sedikit berbeda. Vektor di sana dibuat hanya dari literal 1.0, 2.0, 3.0 tanpa operasi lain yang mengunci tipenya, jadi compiler tidak punya petunjuk apakah itu `f32` atau `f64`. Perbaikannya menulis `let a: Vector3<f64> = Vector3::new(1.0, 2.0, 3.0)`, diterapkan konsisten pada `b`, `p`, `q`, serta `w`. Setelah anotasinya terpasang, cargo test lolos dengan 17 passed dan 0 failed untuk keseluruhan proyek.

Fungsi Statistik

Empat metode bawaan dipanggil pada vektor `a`, yaitu `sum()`, `mean()`, `max()`, dan `min()`, yang berturut-turut memberi 6, 2, 3, dan 1. Task II harus melipat sendiri `max` dan `min` dengan `fold` karena `f64` tidak mengimplementasikan `Ord`, sedangkan di sini cukup satu pemanggilan metode.

Latihan Terapan

Latihan pertama mencetak norma `p` sebesar 5, norma `q` sebesar 4, dot product 12, serta sudut sekitar 53.13 derajat, sudut baku segitiga 3-4-5 yang bisa diperiksa tanpa kalkulator. Latihan kedua membuat `w` berisi 1 sampai 10, dicetak lewat `w.transpose()` supaya tidak menghabiskan sepuluh baris terminal, dengan jumlah 55, rata-rata 5.5, maksimum 10, dan minimum 1. Latihan ketiga tidak dicetak ke layar melainkan dikerjakan sebagai unit test `dot_diri_sendiri_sama_dengan_kuadrat_norma`. Toleransi dipakai di sana karena `norm()` melibatkan akar kuadrat, sehingga hasil kuadratnya belum tentu kembali persis ke 14 dalam representasi biner `f64`.

5.5 Output Program

Keluaran lengkap Task III saat dijalankan dengan cargo run -- 3 terlihat pada gambar berikut.

```
=== Task III: Vektor dengan nalgebra ===  
a =  
     $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$   
  
dimensi a: 3  
b4 =  
     $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$   
  
dimensi b4: 4  
a + b =  
     $\begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$   
  
a - b =  
     $\begin{bmatrix} -3 \\ -3 \\ -3 \end{bmatrix}$ 
```

Gambar 5.1a: Hasil cargo run untuk Task III, bagian atas

```

2a      =

$$\begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$


a o b =

$$\begin{bmatrix} 4 \\ 10 \\ 18 \end{bmatrix}$$


a . b = 32
||a|| = 3.7416573867739413
sudut a dan b = 12.933154491899135 derajat
sum     = 6
mean    = 2
max     = 3
min     = 1
||p|| = 5, ||q|| = 4
p . q = 12
sudut p dan q = 53.13010235415599 derajat
w =

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{bmatrix}$$


jumlah w      = 55
rata-rata w   = 5.5
maksimum w    = 10
minimum w     = 1

```

```
~/Doc/Ku/aljabar-linear/2/praktikum-al on main !1 ?5
```

Gambar 5.1b: Hasil cargo run untuk Task III, bagian bawah

Vektor dicetak nalgebra sebagai kolom berkurung sehingga satu vektor memakan beberapa baris terminal, kecuali w yang sengaja ditranspos. Blok aritmetika memberi [5, 7, 9], [-3, -3, -3], [2, 4, 6], dan [4, 10, 18], persis seperti kolom hasil pada Tabel 6 modul, lalu menyusul $a \cdot b$ sama dengan 32, norma a sekitar 3.742, dan sudut sekitar 12.93 derajat.

5.6 Perbandingan dengan Implementasi Manual

Tabel 4 modul menempatkan array native dan SVector berhadapan langsung, dan kedua task ini menunjukkan bedanya di kode nyata. Pada Task II, baris $u + v$ harus dikomentari karena rustc menolaknya dengan error[E0369] cannot add [f64; 3] to [f64; 3], begitu pula $k * a$ yang tidak punya implementasi Mul untuk array. Akibatnya setiap operasi ditulis sebagai fungsi terpisah dengan `assert_eq!` sendiri untuk mencocokkan panjang slice. Pemeriksaan itu baru berjalan saat runtime, jadi ketidakcocokan dimensi lolos kompilasi dan baru ketahuan ketika program panik.

Task III membalik keadaan itu. SVector mendukung $a + b$ dan $2.0 * a$ lewat operator karena trait aritmetiknya sudah diimplementasikan crate, dan dimensi menjadi bagian dari tipe sehingga menjumlahkan Vector3 dengan SVector<f64, 4> ditolak sebelum program sempat jalan. Tidak ada satu pun `assert_eq!` panjang di `src/task3.rs`, pekerjaan itu sudah diambil alih sistem tipe. Tabel 4 juga mencatat performa keduanya setara berkat zero-cost abstraction, sebab generik dan trait nalgebra habis saat monomorfisasi.

5.7 Lesson Learnt

Keamanan tipe di nalgebra bekerja karena dimensi ikut masuk ke dalam tipe, bukan disimpan sebagai angka di dalam struct. Menggeser pemeriksaan dari runtime ke compile time menghapus satu kelas kesalahan tanpa menambah sebaris pun kode pemeriksaan.

Keringkasannya juga terasa. Task II butuh tujuh fungsi bebas untuk penjumlahan, perkalian skalar, dot product, dan empat statistiknya, sedangkan Task III mengerjakan hal yang sama tanpa satu pun fungsi pembantu. Kode yang tidak ditulis tidak perlu diuji dan tidak bisa salah, jadi pengujian di sini fokus pada nilai hasilnya, bukan implementasi operasinya. Imbalannya datang bersama sistem tipe generik yang sesekali menuntut anotasi eksplisit seperti E0282 tadi, dan itu jauh lebih murah dibanding menulis serta menguji seluruh operasi vektor dari nol.

BAB VI PENGUJIAN OTOMATIS DAN KUALITAS KODE

6.1 Unit Test

Atribut `#[cfg(test)]` di atas mod `tests` membuat isi modul itu hanya ikut dikompilasi ketika cargo test dijalankan. Pada cargo build maupun cargo run kompilator melewatinya, jadi biner hasil build tidak membawa satu baris pun kode uji. Efek sampingnya enak, pembantu khusus test seperti konstanta `EPS` dan fungsi dekat di `task2.rs` boleh menganggur di sana tanpa memicu peringatan `dead_code`. Baris `use super::*` di awal modul menarik seluruh item dari modul induk, jadi fungsi yang tidak ditandai pub sekalipun, seperti `klasifikasi` dan `hampir_sama` di `task1.rs`, tetap terlihat dari dalam test.

Tujuh belas test tersebar di tiga berkas, lima di `task1.rs` untuk klasifikasi ganjil genap, perbandingan float, aritmetika, dan slicing, enam di `task2.rs` untuk operasi vektor manual, dan enam lagi di `task3.rs` untuk tipe nalgebra. Dua di antaranya ada karena modul memintanya. Test `dot_diri_sendiri_sama_dengan_kuadrat_norma` menjawab poin E.3 Task III, memverifikasi identitas $u \cdot u$ sama dengan kuadrat norma u dengan membandingkan `a.dot(&a)` terhadap `a.norm().powi(2)`, dan identitas itu benar secara matematis tetapi jalur hitungnya berbeda karena sisi kanan sempat melewati `sqrt` lalu dikuadratkan lagi. Test `panjang_beda_ditolak` memakai atribut `should_panic` untuk membuktikan `assert_eq!` penjaga panjang di dalam `dot` memang memicu panic ketika kedua slice tidak sama panjang, bukan diam-diam memangkas hasil mengikuti slice terpendek seperti perilaku `zip` tanpa penjaga. Argumen `expected` mencocokkan pesan panic-nya, sehingga test tetap gagal seandainya panic datang dari sebab lain. Hampir semua `assert` ditulis sebagai selisih mutlak terhadap ambang $1e-10$ sampai $1e-12$, bukan `assert_eq!` langsung, sebab `f64` menyimpan $0.1 + 0.2$ sebagai `0.30000000000000004` dan perbandingan persis akan gagal padahal rumusnya benar. `assert_eq!` hanya bertahan di tempat yang nilainya eksak, yaitu perbandingan `&str`, `i32`, dan `Vector3` berisi bilangan bulat kecil. Satu ambang sengaja dilonggarkan ke $1e-4$ pada test sudut `p` dan `q`, karena angka acuannya sendiri, `53.1301` derajat, sudah dipotong empat desimal.

```
#[test]
#[should_panic(expected = "panjang kedua vektor harus sama")]
fn panjang_beda_ditolak() {
    dot(&[1.0, 2.0, 3.0], &[1.0, 1.0]);
}
```

6.2 Hasil Pengujian

Seluruh test dijalankan sekaligus lewat perintah `cargo test` dari akar proyek.


```
praktikum-al — ghaisan@Ghaisans-MacBook-Pro — ../praktikum-al — -zsh — 92x32

Finished `test` profile [unoptimized + debuginfo] target(s) in 0.01s
Running unittests src/main.rs (target/debug/deps/praktikum_al-259a931b5ecf0107)

running 17 tests
test task1::tests::hampir_sama_menoleransi_galat_float ... ok
test task1::tests::indexing_dan_slicing ... ok
test task1::tests::operasi_float_dasar ... ok
test task1::tests::klasifikasi_genap_ganjil ... ok
test task1::tests::pembagian_i32_terpotong ... ok
test task2::tests::dot_ujian ... ok
test task2::tests::skalar_ujian ... ok
test task2::tests::statistik_nilai_negatif ... ok
test task2::tests::statistik_ujian ... ok
test task2::tests::tambah_ujian ... ok
test task3::tests::dot_diri_sendiri_sama_dengan_kuadrat_norma ... ok
test task3::tests::dot_product_dan_norma ... ok
test task3::tests::norma_dan_sudut_p_q ... ok
test task3::tests::operasi_element_wise ... ok
test task3::tests::statistik_vektor_a ... ok
test task2::tests::panjang_beda_ditolak - should panic ... ok
test task3::tests::statistik_vektor_w ... ok

test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

~/Doc/Kuliah/aljabar-linear/2026-09-07-modul-1/praktikum-al on main !1 ?5 at 09:45:13
> 
```

Gambar 6.1: Hasil cargo test, 17 test lolos

Keluarannya ditutup baris test result: ok. 17 passed; 0 failed. Urutan test di layar tidak mengikuti urutan berkas karena cargo menjalankannya paralel di beberapa thread, sementara baris panjang_beda_ditolak muncul dengan penanda should panic, tanda ia lolos justru karena programnya panic seperti yang diharapkan. Waktu eksekusi terbaca 0.00 detik, wajar untuk vektor yang dimensi terbesarnya sepuluh. Tidak ada test yang ditandai ignored maupun filtered out, jadi angka 17 itu benar-benar seluruh test yang ditulis.

6.3 Kualitas Kode

Di luar kebenaran hasil, kode dilewatkan dua alat bawaan toolchain. Perintah cargo build selesai tanpa satu pun warning, cargo clippy --all-targets bersih dan hanya mencetak baris Checking serta Finished, dan cargo fmt --check berhenti tanpa diff. Bendera --all-targets dipakai supaya linter ikut masuk ke modul di dalam #[cfg(test)], sebab tanpa itu kode uji luput dari pemeriksaan. Satu-satunya penekanan lint di seluruh proyek adalah #[allow(clippy::approx_constant)] di atas binding phi pada task1.rs, dan itu pun dicontohkan modul karena clippy membaca 3.14159 sebagai pendekatan kasar dari PI. Ketiga pemeriksaan itu menjawab rubrik Tabel 7 secara langsung. Kriteria Setup Environment berbobot 20 persen memberi nilai A untuk toolchain yang terkonfigurasi sempurna dengan cargo build tanpa warning, terpenuhi memakai rustc 1.98.1, cargo 1.98.1, dan nalgebra 0.35.0 pada edition 2024, sedangkan kriteria Dasar Sintaks dan Sistem Tipe yang juga berbobot 20 persen mensyaratkan kode lolos cargo clippy, dan hasil bersih pada seluruh target memenuhinya.

BAB VII KESIMPULAN

Keempat task Modul 1 selesai dan seluruhnya dijalankan di mesin sendiri, MacBook Pro M2 Pro, dengan rustc 1.98.1, cargo 1.98.1, edition 2024, dan nalgebra 0.35.0. Task 0 sendiri hanya memastikan crate nalgebra sudah terpasang dan bisa dikompilasi, jadi tiga task sesudahnya berangkat dari lingkungan yang terbukti jalan. Proyek lolos cargo build tanpa satu pun warning, clippy bersih, cargo fmt --check tidak menemukan selisih format, dan cargo test menghasilkan 17 test lolos dengan 0 gagal, terbagi lima di task1.rs serta enam masing-masing di task2.rs dan task3.rs. Empat kriteria rubrik modul, dari setup toolchain sampai perbandingan vektor manual dengan nalgebra, karena itu terpenuhi dengan angka yang bisa diperiksa ulang siapa pun yang menjalankan proyeknya.

Yang paling terasa dari Rust adalah berapa banyak kesalahan yang tidak sempat jadi bug runtime. Binding immutable secara default, sehingga mengubah nilai variabel tanpa mut berhenti di layar compiler, dan pemeriksaan tipe yang statis membuat $7/2$ pada i32 tetap 3 sementara `.sqrt()` sama sekali tidak tersedia untuk integer sampai nilainya di-cast ke f64. Jebakan yang paling berkesan justru muncul di Task III. Hasil pembagian `a.dot(&b) / (a.norm() * b.norm())` masih generik, jadi tanpa anotasi `let cos: f64` pemanggilan `.acos()` langsung ditolak dengan E0282 type annotations needed, sebuah galat yang di bahasa dinamis baru muncul sebagai nilai aneh di tengah perhitungan sudut. Sebagian besar waktu perbaikan di modul ini habis untuk membaca pesan compiler, bukan menelusuri keluaran program yang sudah terlanjur salah.

Dua cara merepresentasikan vektor yang dicoba di modul ini saling melengkapi. Array native menunjukkan apa yang tidak disediakan bahasa: `let w = u + v;` ditolak dengan E0369 cannot add `[f64; 3]` to `[f64; 3]`, jadi penjumlahan, perkalian skalar, dot product, sampai fungsi statistik ditulis sendiri lewat `zip`, `map`, dan `fold`. Pemeriksaan dimensi ikut jatuh ke tangan programmer karena `zip` diam-diam berhenti di slice terpendek, dan f64 yang tidak mengimplementasikan Ord gara-gara NaN memaksa maks dan mini ditulis dengan `f64::max` serta `f64::min`. Di nalgebra kode yang sama menyusut drastis, `a + b` dan `2.0 * a` bisa ditulis apa adanya, panjang vektor terkunci di tipe lewat `Vector3` dan `SVector<f64, 10>`, sehingga `assert` dimensi manual tidak diperlukan lagi. Hasil numeriknya tetap cocok dengan hitungan tangan, dot product a dan b sama-sama 32, norma a bernilai 3.7416573867739413, dan sudut p dan q keluar 53.13010235415599 derajat. Harga dari kenyamanan itu adalah anotasi tipe eksplisit di beberapa tempat.

Modul berikutnya bergerak ke operasi vektor lanjutan dan matriks, tempat jumlah dimensi yang bisa salah naik jauh dibanding vektor tiga elemen. Pola kerja modul ini, tulis versi manualnya dulu, ganti dengan nalgebra, lalu kunci hasilnya dengan unit test bertoleransi, mestinya lebih terasa gunanya di sana.